

Components, Props & State

• Functional components: (QusetionCard)

A functional component is just a JavaScript function that returns JSX. That means in the return we use the javascript `<h1> ....`

• Props (passing data between components): (ProgressBar)

Functional components receive input data through props, which are objects containing key-value pairs. Two functions one is the parent that has data and the child where it takes the data we use `{}` to pass the data. Here we passed the `current + total`

```
/* props: here we paased the variables that where defined in the parent (QustionsCard) */
export default function ProgressBar({ current, total }: ProgressBarProps) {
  const pct = (current / total) * 100
```

• useState hook (managing component state) : (QuizPage)

Add , track, update the data inside a functional component

```
import React, { useState } from 'react';
import './App.css'

const App = () => {
  const [num, setNum] = useState(0);
  const handleClick = () => {
    setNum(num + 1);
  };

  return (
    <div className="App">
      <h2> {num}</h2>
      <button onClick={handleClick}>
        Add one
      </button>
    </div>
  );
};

export default App;
```

In this example we use it to increase the number by 1 when clicking on the button
Another example from my react project :

```
import { useState } from 'react' 4.6k (gzipped: 1.9k)
import { useNavigate } from 'react-router-dom' 210k (gzipped: 65.8k)
import { questions } from '../data/questions'
import Navbar from '../components/Navbar'
import Footer from '../components/Footer'
import QuestionCard from '../components/QuestionCard'

export default function QuizPage() {
  const navigate = useNavigate()
  const [current, setCurrent] = useState(0) /* which Q u are in now ? */
  const [score, setScore] = useState(0)
  const [selected, setSelected] = useState<string | null>(null)
```

• Event handling & Conditional rendering in React (used OnClick: NavBar)

React's approach to event handling is similar to plain JavaScript but with a few important differences.

1. We use camelCase
2. **Synthetic Events:** Events in React are wrapped in a cross-browser wrapper called synthetic events it make sure it works in safari and chrome
3. Passing arguments: you can pass arguments

Conditional rendering:(ResultPage) if statement, ternary operator ? and the &&

• Key concept: Component reusability (AnswerButton, Nav ,Footer)

Once you have reusable components, you can compose them together to build more complex UIs. we use it by importing the file where we used it

```
// Button.js
import React from 'react';
const Button = ({ text, onClick }) => {
  return (
    <button onClick={onClick}>
      {text}
    </button>
  );
};
export default Button;
```

```
// App.js
import React from 'react';
import Button from './Button';
const App = () => {
  const handleClick = () => {
    alert('Button clicked!');
  };
  return (
    <div>
      <h1>Hello, React.js!</h1>
      <Button text="Click Me" onClick={handleClick} />
    </div>
  );
};
export default App;
```

Lists, Forms & Events

• Rendering lists with .map() (QuestionsCard):

This line first builds the array by spreading the 3 wrong answers and adding the correct one, then .sort() shuffles them alphabetically so the correct answer isn't always last.

Is used to loop over arrays then returns a **new array** after performing something on each item.

Simple example:

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6, 8]
```

Keys in React & Controlled components (forms)

Keys help React **identify which items changed, were added, or removed** in a list. With keys, it only updates what changed much more efficiently.

A **controlled component** is an input whose value is **controlled by React state, not the DOM**.

```
function ControlledForm() {
  // 1. Initialize React state to hold the input value
  const [name, setName] = useState('');

  const handleChange = (event) => {
    // 2. Update React state on every keystroke
    setName(event.target.value);
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`Submitted Name: ${name}`);
  };
}
```

Controlled component: React controls the value.

Uncontrolled component: The browser (DOM) controls the value.

Form submission handling (Home page)

We use the controlled components we handle things for example in my project I used it when u press start without entering ur name it prevents you and give u a message

Input validation

Used it in my homepage on the handleSubmit > where you can't submit a whitespace or a number it's only a name

```
function handleSubmit(e: { preventDefault: () => void }) {
  e.preventDefault()
  if (name.trim() === "") {
    setError("Please enter your name to start! 🖱️")
    return
  }
  if (/\\d/.test(name)) {
    setError("Name can't contain numbers! 🚫")
    return
  }
  navigate('/quiz', { state: { name: name.trim() } })
}
```

Multiple input handling

Used it in the form where we handle the name + age

Why do we use Rem rather than Px ?

rem scales with the user's browser font size setting, so it's more accessible and flexible than the fixed, user-ignoring **px**.

If a user makes their browser text bigger for easier reading, rem respects that but px doesn't.